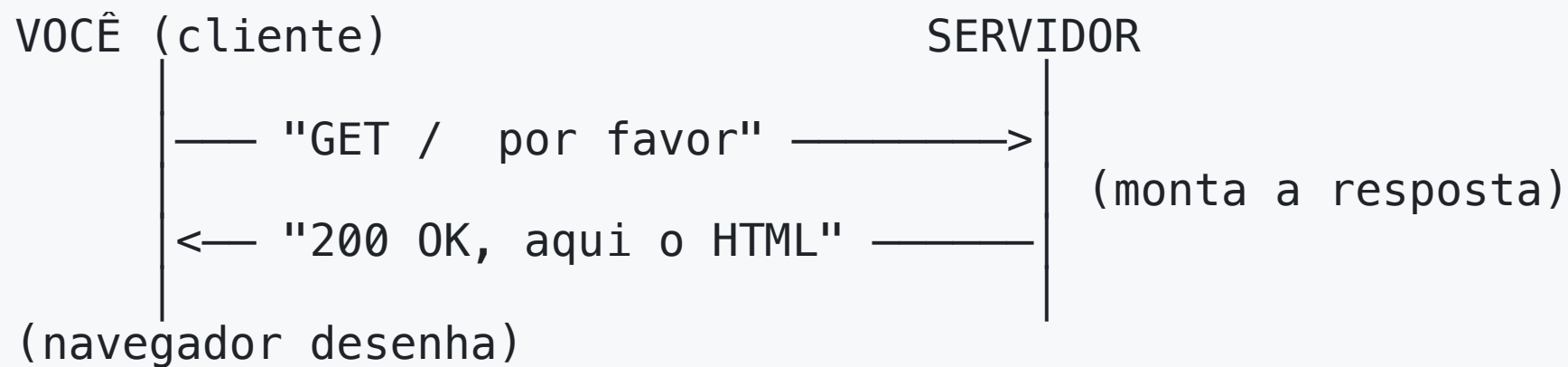


Aula 04

HTTP e o primeiro servidor

Hoje você escreve o programa que fica **do outro lado**

O que acontece ao acessar um site



Isso é HTTP: pergunta e resposta

3 características do HTTP

1. O cliente sempre inicia

O servidor nunca liga primeiro

2. Cada requisição é independente

O servidor não "lembra" da anterior (*stateless*)

3. É texto puro

Literalmente texto trafegando pela rede

A analogia do balcão

Você chega e faz **um pedido específico**

→ "um X-burger, sem cebola"

Recebe **uma resposta**

→ o lanche, ou "acabou o pão"

A conversa **termina**.

Quer mais? Faz outro pedido.

Anatomia da requisição

```
POST /tarefas HTTP/1.1      ← método + caminho
Host: localhost:3000        ← cabeçalhos
Content-Type: application/json
{ "titulo": "Estudar Node" } ← corpo
```

Anatomia da resposta

```
HTTP/1.1 201 Created      ← status  
Content-Type: application/json ← cabeçalhos  
  
{ "id": 1, "titulo": "Estudar Node" } ← corpo
```

Os métodos = os verbos

Método	Intenção
GET	buscar / ler
POST	criar
PUT	substituir inteiro
PATCH	alterar em parte
DELETE	remover

⚠ A barra de endereços só faz **GET**

Status codes: a primeira casa já diz tudo

Faixa	Significado
2xx	deu certo — "toma aqui"
3xx	está em outro lugar
4xx	you errou o pedido
5xx	eu quebrei, desculpa

Os que você vai usar toda semana

200	OK	deu certo
201	Created	criou (POST)
204	No Content	ok, sem corpo
400	Bad Request	dados inválidos
404	Not Found	não existe
500	Server Error	seu código quebrou

4xx ou 5xx? A diferença importa

4xx = culpa de quem **pediu**

5xx = culpa de quem **respondeu**

Devolver **500** para um dado inválido
esconde o erro do cliente.

E vice-versa.

Servidor com o Node puro

```
import { createServer } from "node:http";

const servidor = createServer((req, res) => {
  res.writeHead(200, {
    "Content-Type": "text/plain; charset=utf-8"
  });
  res.end("Olá, mundo!");
});

servidor.listen(3000);
```

Você acabou de escrever um servidor web 🎉

Repare: o programa não termina

Ele fica ouvindo a porta 3000,
esperando pedidos chegarem.

Para parar: `Ctrl+C`

Mas e com várias rotas?

```
createServer((req, res) => {  
  if (req.url === "/" && req.method === "GET") {...}  
  else if (req.url === "/tarefas") {...}  
  else if (req.url.startsWith("/tarefas/")) {  
    const id = req.url.split("/")[2]; // 🤔  
  }  
  else if (req.method === "POST") {  
    // 🧑 ler o corpo pedaço por pedaço  
  }  
});
```

Foi essa dor que criou o Express

```
npm install express
```

```
import express from "express";  
const app = express();  
  
app.get("/", (req, res) => {  
  res.send("Olá, mundo!");  
});  
  
app.listen(3000);
```

As 3 partes de toda rota

```
app.get("/tarefas", (req, res) => { ... });
```

The diagram consists of four vertical arrows pointing upwards from the comments on the right to the corresponding parts of the code on the left. The first arrow points from 'o método' to 'get'. The second arrow points from 'o caminho' to '/tarefas'. The third arrow points from 'o que o cliente PEDIU' to 'req'. The fourth arrow points from 'o que EU respondo' to 'res'.

- o que EU respondo
- o que o cliente PEDIU
- o caminho
- o método

`req` e `res` são convenção — o mundo todo usa assim.

O que tem dentro do req

Propriedade	O que é
req.method	"GET" , "POST" ...
req.originalUrl	o caminho pedido
req.params	os :parametros da rota
req.query	o que vem depois do ?
req.body	os dados enviados (aula 05)

Todos são **objetos** comuns.

Você acessa com `.` ou desestrutura.

Formas de responder

```
res.send("texto");           // texto/HTML
res.json({ ok: true });      // JSON ★
res.status(404).json({ erro: "..."}); // status + JSON
res.status(204).end();        // sem corpo
```

`res.status()` devolve o próprio `res` → dá para encadear

Os métodos do `res`

Método	O que faz
<code>res.json(obj)</code>	converte para JSON e envia ★
<code>res.send(texto)</code>	envia texto ou HTML
<code>res.status(n)</code>	define o status (não envia)
<code>res.end()</code>	encerra sem corpo

⚠ `res.status(404)` sozinho **não responde nada**.

Ele só define o número. Precisa de `.json()` ou `.end()` depois.

! Uma requisição = UMA resposta

```
if (!tarefa) {  
  return res.status(404).json({ erro: "não achei" });  
}           ↑  
// esse return é OBRIGATÓRIO  
  
res.json(tarefa);
```

Sem ele:

Cannot set headers after they are sent

`req.params` — qual recurso

```
app.get("/tarefas/:id", (req, res) => {  
  const { id } = req.params;  
  res.json({ id });  
});
```

GET /tarefas/42 → { "id": "42" }

⚠ **Sempre texto!** `"42" === 42` é `false`

Converta: `Number(id)`

`Number()` e `Number.isNaN()`

```
Number("42")           // 42
Number("3.14")          // 3.14
Number("abc")           // NaN    ← "Not a Number"
Number("")              // 0      ← cuidado!
```

```
Number.isNaN(Number("abc")) // true  → conversão falhou
Number.isInteger(42.5)      // false
```

Por que `Number.isNaN` existe?

Porque `NaN` é bizarro:

```
NaN === NaN // false 🤯
```

Ele não é igual nem a si mesmo.

```
Number.isNaN(valor) // ✅ a forma correta de testar
```

No curso: validar id que vem da URL

→ se for `NaN`, responde `400`.

`req.query` — o que vem depois do `?`

```
app.get("/tarefas", (req, res) => {  
  const { concluida, busca } = req.query;  
});
```

GET /tarefas?concluida=true&busca=node

→ { concluida: "true", busca: "node" }

! "true" é **string**, não booleano

Quando usar cada um

params	qual recurso → /tarefas/5
query	como filtrar → ?ordem=asc

Testando sem sair do editor

Extensão **REST Client** → arquivo `api.http`

```
### Listar
GET http://localhost:3000/tarefas

### Criar
POST http://localhost:3000/tarefas
Content-Type: application/json

{ "titulo": "Estudar Express" }
```

Um botão "Send Request" aparece acima de cada bloco.

Pare de reiniciar na mão

```
"scripts": {  
  "dev": "node --watch server.js"  
}
```

```
npm run dev
```

Salvou → reiniciou. 🎉

Recapitulando

1. HTTP: método + caminho vão, status + corpo voltam
2. Express organiza rotas e cuida do trabalho chato
3. `params` = qual · `query` = como · **ambos são texto**

Mão na massa

`aula-04/exercicios/README.md`

Próxima: criar, atualizar e apagar — o **CRUD**